Appendix A: Code for maximum

Here we give the Mathematica code used for the rigorous verification of the Palacios conjecture for the maximum. It explicitly checks whether any graph attaining the maximal inverse degree index also attains the maximal symmetric division deg index, by intersecting the sets of extremal graphs.

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------All c-cyclic graphs with Delta=n-1----------*)
AllGraphs[n_Integer,c_Integer]:=Module[{base,others,allExtraEdges,extraSubsets},
others=Range[2,n];
base=Table[{1,i},{i,others}];
allExtraEdges=Subsets[others,{2}];
extraSubsets=Subsets[allExtraEdges,{c}];
Map[Join[base,#]&,extraSubsets]];

(*----------Explicit construction of Bianchi maximal graphs----------*)
BianchiGraph[c_Integer,n_Integer,k_Integer]:=Module[{u=1,edges,v,w,x,y},
Which[
c==1,(*H_{n,1}*)v=2;w={3};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]}}],
c==2,(*H_{n,2}*)v=2;w={3,4};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]}}],
c==3,If[k==1,(*H_{n,3}:4,2,2,2*)v=2;w=Range[3,5];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,3}]],
(*k=2:triangle (3,3,3)*)v={2,3,4};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[2]],v[[3]]},{v[[3]],v[[1]]}}]],
c==4,If[k==1,(*H_{n,4}:5,2,2,2,2*)v=2;w=Range[3,6];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,4}]],
(*k=2:4,3,3,2*)v=2;w={3,4};x=5;
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x}}]],
c==5,If[k==1,(*6,2^5*)v=2;w=Range[3,7];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,5}]],
If[k==2,(*5,3,3,2,2*)v=2;w={3,4};x={5,6};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x[[1]]},{v,x[[2]]}}],
(*k=3:4,4,3,3*)v={2,3};w={4,5};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[1]],w[[1]]},{v[[1]],w[[2]]},{v[[2]],
  w[[1]]},{v[[2]],w[[2]]}}]],
c==6,If[k==1,(*7,2^6*)v=2;w=Range[3,8];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,6}]],
If[k==2,(*6,3,3,2,2,2*)v=2;w={3,4};x={5,6,7};
```

```
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x[[1]]},{v,x[[2]]},
 {v,x[[3]]}}],
If[k==3,(*5,4,3,3,2 { corrected*)v=2; w={3,4,5,6};
edges=Join[Table[{u,i},{i,2,n}],{{2,3},{2,4},{2,5},{2,6},{3,4},{3,5}}],
(*k=4:4,4,4,4*)v={2,3,4,5};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[1]],v[[3]]},{v[[1]],v[[4]]},{v[[2]],
  v[[3]]},{v[[2]],v[[4]]},{v[[3]],v[[4]]}}]]]];
edges];

(*----------Verification of the conjecture----------*)
CheckConjecture[c_Integer,nMax_Integer]:=Module[{n,graphs,iVals,sVals,
 maxI,maxS,maxIgraphs,maxSgraphs,vecs,vecGraphs,vecI,vecS,sameMaxGraphQ},
Print["========== c = ",c," =========="];
Do[If[n<c+2,Continue[]];
graphs=AllGraphs[n,c];
If[Length[graphs]==0,Continue[]];
iVals=Iindex/@graphs;
sVals=SDD/@graphs;
maxI=Max[iVals];
maxS=Max[sVals];
maxIgraphs=Pick[graphs,iVals,maxI];
maxSgraphs=Pick[graphs,sVals,maxS];
Print["n=",n," (",Length[graphs]," graphs)"];
Print["  max I = ",maxI," = ",Rationalize[maxI]];
Print["  max SDD = ",maxS," = ",Rationalize[maxS]];
(*Definition of Bianchi vectors (degree sequences)*)
vecs=Which[
c==1,{{n-1,2,2}~Join~Table[1,{n-3}]},
c==2,{{n-1,3,2,2}~Join~Table[1,{n-4}]},
c==3,{{n-1,4,2,2,2}~Join~Table[1,{n-5}],{n-1,3,3,3}~Join~Table[1,{n-4}]},
c==4,{{n-1,5,2,2,2,2}~Join~Table[1,{n-6}],{n-1,4,3,3,2}~Join~Table[1,{n-5}]},
c==5,{{n-1,6,2,2,2,2,2}~Join~Table[1,{n-7}],{n-1,5,3,3,2,2}~Join~Table[1,{n-6}],
 {n-1,4,4,3,3}~Join~Table[1,{n-5}]},
c==6,{{n-1,7,2,2,2,2,2,2}~Join~Table[1,{n-8}],{n-1,6,3,3,2,2,2}~Join~Table[1,{n-7}],
 {n-1,5,4,3,3,2}~Join~Table[1,{n-6}],{n-1,4,4,4,4}~Join~Table[1,{n-5}]}
];
vecGraphs=Table[BianchiGraph[c,n,k],{k,1,Length[vecs]}];
vecI=Iindex/@vecGraphs;
vecS=SDD/@vecGraphs;
Print["  Values for Bianchi maximal vectors:"];
Do[
Print["    Vector ",i,": ",vecs[[i]],"  I=",vecI[[i]],"  SDD=",vecS[[i]]];
If[vecI[[i]]==maxI,Print["      <-- attains max I"]];
If[vecS[[i]]==maxS,Print["      <-- attains max SDD"]];,
{i,Length[vecs]}
];
sameMaxGraphQ=Length[Intersection[maxIgraphs,maxSgraphs]]>0;
Print["  Max I and Max SDD on the same graph? ",sameMaxGraphQ];
If[!sameMaxGraphQ,
Print["  *** WARNING: Palacios conjecture DOES NOT HOLD for this case!"],
Print["  (Conjecture currently holds.)"]
];
Print[""];,
{n,c+2,nMax}
]
];

(*----------Execution----------*)
Do[CheckConjecture[c,c+5],{c,1,6}]
```

Appendix B: Code for minimum

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity of graph given by edge list----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue=!={},v=First[queue];
queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------All connected graphs with n vertices and m edges----------*)
AllConnectedGraphs[n_Integer,m_Integer]:=Module[{allEdges,edgeSets},
allEdges=Subsets[Range[n],{2}];
edgeSets=Subsets[allEdges,{m}];
Select[edgeSets,ConnectedQ[#,n]&]];

(*----------All c-cyclic graphs of order n----------*)
AllCCyclicGraphs[n_Integer,c_Integer]:=AllConnectedGraphs[n,n+c-1];

(*----------Degree sequence sorted descending----------*)
DegreeSequence[edges_List,n_Integer]:=Sort[VertexDegrees[edges,n],Greater];

(*----------Maximum number of subsets we allow----------*)
maxSubsets=500000;

(*----------Main routine for checking minimum----------*)
CheckMinimumConjecture[c_Integer,nMax_Integer]:=
Module[{n,m,totalSubsets,graphs,iVals,sVals,minI,minS,
 minIpos,minSpos,seqI,seqS,sI},
Print["==================== c = ",c," ===================="];
Do[m=n+c-1;
totalSubsets=Binomial[Binomial[n,2],m];
If[totalSubsets>maxSubsets,
Print["n = ",n," : skipping (",totalSubsets," subsets > ",maxSubsets,")"];
Continue[]];
Print["n = ",n," (",totalSubsets," subsets) ..."];
graphs=AllCCyclicGraphs[n,c];
```

```
Print["  number of connected graphs = ",Length[graphs]];
If[Length[graphs]==0,Continue[]];
iVals=Iindex/@graphs;
sVals=SDD/@graphs;
minI=Min[iVals];
minS=Min[sVals];
minIpos=First[FirstPosition[iVals,minI]];
minSpos=First[FirstPosition[sVals,minS]];
seqI=DegreeSequence[graphs[[minIpos]],n];
seqS=DegreeSequence[graphs[[minSpos]],n];
sI=sVals[[minIpos]];
Print["  min I = ",minI,"   SDD(min I) = ",sI,"   sequence: ",seqI];
Print["  min SDD = ",minS,"   sequence: ",seqS];
If[sI==minS,
Print["  \[Checkmark] Same SDD values \[Dash] minima coincide."],
Print["  \[Cross] DIFFERENT SDD values \[Dash] COUNTEREXAMPLE!"]];
If[seqI==seqS,
Print["  \[Checkmark] Same degree sequence \[Dash] conjecture holds."],
Print["  \[Cross] DIFFERENT degree sequences \[Dash] conjecture DOES NOT HOLD."]];
Print[""];,{n,c+2,nMax}]];

(*----------EXECUTION for c=3,4,5,6----------*)
Do[CheckMinimumConjecture[c,7],{c,3,6}]
```

Appendix C: Code for minimum for $c = 6, n = 8$

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity (BFS)----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue=!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------Backtracking: all realizations of a degree sequence----------*)
AllGraphsFromDegreeSequence[degseq_List]:=
```

```
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDegs,newEdges},
If[AllTrue[deg,#==0&],
AppendTo[results,Sort[edges]];
Return[]
];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDegs=deg;
newDegs[[i]]=0;
newEdges=edges;
Do[newDegs[[v]]=newDegs[[v]]-1;
AppendTo[newEdges,{i,v}],{v,subset}];
If[Min[newDegs]>=0,backtrack[newDegs,newEdges]],
{s,Subsets[others,{d}]}]
];
backtrack[degseq,{}];
results
];

(*----------MAIN PART----------*)
n=8;
c=6;
m=n+c-1;(*13*)

(*All possible degree sequences for n=8,m=13*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..7*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results=!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,4,3,3,3,3,3,3) is 53/2 = ",53/2];
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]
```

Appendix D: Code for minimum for $c = 6, n = 9$

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity (BFS)----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------Backtracking: all realizations of a degree sequence----------*)
AllGraphsFromDegreeSequence[degseq_List]:=
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDegs,newEdges},
If[AllTrue[deg,#==0&],
AppendTo[results,Sort[edges]];
Return[]
];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDegs=deg;
newDegs[[i]]=0;
newEdges=edges;
Do[newDegs[[v]]=newDegs[[v]]-1;
AppendTo[newEdges,{i,v}],{v,subset}];
If[Min[newDegs]>=0,backtrack[newDegs,newEdges]],
{s,Subsets[others,{d}]}]
];
backtrack[degseq,{}];
results
];

(*----------MAIN PART----------*)
```

```
n=9;
c=6;
m=n+c-1;(*14*)
(*All possible degree sequences for n=9,m=14*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..8*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results=!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,3,3,3,3,3,3,3,3) is ",
SDD[AllGraphsFromDegreeSequence[{4,3,3,3,3,3,3,3,3}][[1]]]];
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]
```

Appendix E: Code for automated analysis and transition points for $k = 3$ to 30

It generates all integer partitions of $2k$ with parts in $[1, k]$, selects graphic sequences via the Havel–Hakimi test, maximizes the asymptotic functional $A(H)$, and computes exact SDD formulas and the breakpoint $n_0(k)$ by comparing the optimal subgraph with the baseline pattern $\mathbf{v}_1$.

```
(* ============================================================
Automated analysis for c=3 to 30 (optimal H, exact SDD
formulas and breakpoints) Petojevic,Orlic,Palacios
============================================================*)
ClearAll["Global`*"];

(*----------Basic functions----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
```

```
Total[1/deg]];

SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Havel--Hakimi test----------*)
HavelHakimiQ[seq_List]:=Module[{d=Select[seq,#>0&]},
While[True,
If[Length[d]==0,Return[True]];
d=Sort[d,Greater];
If[d[[1]]>=Length[d],Return[False]];
Do[d[[i+1]]=d[[i+1]]-1,{i,d[[1]]}];
d=Rest[d];
If[Min[d]<0,Return[False]];
]];

(*----------Generation of all connected realizations of a given
degree sequence----------*)
AllConnectedRealizations[degseq_List]:=
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[currentDeg_,currentEdges_,vertex_]:=
Module[{candidates,newDeg,newEdges,valid},
If[vertex>n,
If[AllTrue[currentDeg,#==0&],
AppendTo[results,Sort/@currentEdges]];
Return[]];
If[currentDeg[[vertex]]==0,
backtrack[currentDeg,currentEdges,vertex+1];Return[]];
candidates=Select[Range[vertex+1,n],currentDeg[[#]]>0&];
If[Length[candidates]<currentDeg[[vertex]],Return[]];
Do[newDeg=currentDeg;
newEdges=currentEdges;
valid=True;
Do[newDeg[[v]]-=1;
AppendTo[newEdges,{vertex,v}];
If[newDeg[[v]]<0,valid=False;Break[]],{v,subset}];
If[valid,newDeg[[vertex]]=0;
backtrack[newDeg,newEdges,vertex+1]],
{subset,Subsets[candidates,{currentDeg[[vertex]]}]}];
];
backtrack[degseq,{},1];
(*Connectivity check*)
ConnectedQ[edgelist_]:=
Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edgelist}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue=!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;
AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];
Select[results,ConnectedQ]];

(*----------Degree sequences of subgraph H----------*)
HDegreeSequences[c_Integer]:=Module[{parts},
```

```
parts=IntegerPartitions[2 c,All,Range[c]];
Sort[#,Greater]&/@parts];

Afunctional[degrees_List]:=Module[{d=degrees,k},
k=Length[d];Total[1/(1+d)]-k];

CandidateSequences[degrees_List,n_Integer]:=
Module[{k=Length[degrees],leaves},
leaves=Max[0,n-1-k];
Join[{n-1},(1+#)&/@degrees,ConstantArray[1,leaves]]];

(*----------SDD for a specific H (edge list) and given n----------*)
SDDfromH[Hedges_List,hSize_Integer,n_Integer]:=
Module[{leafCount,degH,degG,sdd=0,i,j,a,b,universalDegree=n-1},
leafCount=n-1-hSize;
degH=VertexDegrees[Hedges,hSize];
degG=Join[{universalDegree},(1+#)&/@degH,
ConstantArray[1,leafCount]];
(*edges between universal vertex and H*)
Do[a=degG[[1]];
b=degG[[1+i]];
sdd+=a/b+b/a,{i,hSize}];
(*edges between universal vertex and leaves*)
If[leafCount>0,
Do[a=degG[[1]];
b=degG[[1+hSize+j]];
sdd+=a/b+b/a,{j,leafCount}]];
(*edges inside H*)
Do[a=degG[[1+e[[1]]]];
b=degG[[1+e[[2]]]];
sdd+=a/b+b/a,{e,Hedges}];
sdd];

(*----------SDD for v1 pattern (universal vertex + c leaves of
degree 2 + remaining leaves)----------*)
SDDv1[c_,n_]:=Module[{hEdges},
hEdges=Table[{1,i},{i,2,c+1}];(*central vertex of degree c,
c leaves*)
SDDfromH[hEdges,c+1,n]];

(*----------Main loop----------*)
Do[
Print["====================================="];
Print["c = ",c];
Print["====================================="];
hSeqs=HDegreeSequences[c];
Print["Number of partitions: ",Length[hSeqs]];
candidates={};
Do[seq=hSeqs[[i]];
If[HavelHakimiQ[seq],
AppendTo[candidates,{seq,Afunctional[seq]}]],
{i,Length[hSeqs]}];
Print["Graphic sequences: ",Length[candidates]];
maxA=Max[candidates[[All,2]]];
bestSeqs=Select[candidates,#[[2]]==maxA&];
Print["Maximum A(H) = ",maxA];
Print["Optimal sequence(s) for H:"];
Do[Print[bestSeqs[[i,1]]],{i,Min[Length[bestSeqs],3]}];
bestSeq=bestSeqs[[1,1]];
(*Find a realization with MAXIMAL internal SDD*)
realizations=AllConnectedRealizations[bestSeq];
```

```
If[Length[realizations]==0,
Print["No connected realization! Skipping."];Continue[]];
internalSDD[edges_]:=Total[Table[With[{a=bestSeq[[e[[1]]]],
 b=bestSeq[[e[[2]]]]},a/b+b/a],{e,edges}]];
bestReal=First[MaximalBy[realizations,internalSDD]];
Print["Chosen realization of H (edges): ",bestReal];
Print["G (for n = ",c+2,"): ",
CandidateSequences[bestSeq,c+2]];
(*Compute SDD for three consecutive values of n*)
n1=c+2;n2=c+3;n3=c+4;
sdd1=SDDfromH[bestReal,Length[bestSeq],n1];
sdd2=SDDfromH[bestReal,Length[bestSeq],n2];
sdd3=SDDfromH[bestReal,Length[bestSeq],n3];
(*For v1*)
v11=SDDv1[c,n1];
v12=SDDv1[c,n2];
v13=SDDv1[c,n3];
(*SDD has the form: n^2 + a*n + b + (n+2c-1)/(n-1)*)
rat[n_]:=(n+2 c-1)/(n-1);
r1=rat[n1];r2=rat[n2];r3=rat[n3];
(*Determine a and b for the new pattern*)
eq1=sdd1-r1-n1^2==a*n1+b;
eq2=sdd2-r2-n2^2==a*n2+b;
sol=Solve[{eq1,eq2},{a,b}];
aNew=a/.sol[[1]];
bNew=b/.sol[[1]];
(*Similarly for v1*)
eq1v=v11-r1-n1^2==aV*n1+bV;
eq2v=v12-r2-n2^2==aV*n2+bV;
solV=Solve[{eq1v,eq2v},{aV,bV}];
aV1=aV/.solV[[1]];
bV1=bV/.solV[[1]];
(*Difference*)
diffA=aNew-aV1;
diffB=bNew-bV1;
Print["Difference SDD(new) - SDD(v1) = ",diffA," * n + ",diffB];
(*Breakpoint*)
If[diffA>0,
n0=Ceiling[-diffB/diffA];
(*Verify that this is indeed the breakpoint*)
If[Chop[SDDfromH[bestReal,Length[bestSeq],n0]-
SDDv1[c,n0]]<=0,n0++];
Print["Breakpoint n0 = ",n0],
Print["The new pattern never dominates v1 (or is always better)."]
];
(*Values of A for the old patterns*)
nSmall=c+2;
If[c>=5,
Module[{hDeg3},
hDeg3=Join[{c-2},{3},{2,2},ConstantArray[1,c-5]];
Print["v3 A = ",Afunctional[hDeg3]];];
Print["v2 A = ",Afunctional[Join[{c-1},{2,2},
ConstantArray[1,c-3]]]];
Print["v1 A = ",Afunctional[Join[{c},ConstantArray[1,c]]]];
Print[""],
{c,3,30}]
```